

DamageTriage: Real-Time Building Damage Segmentation

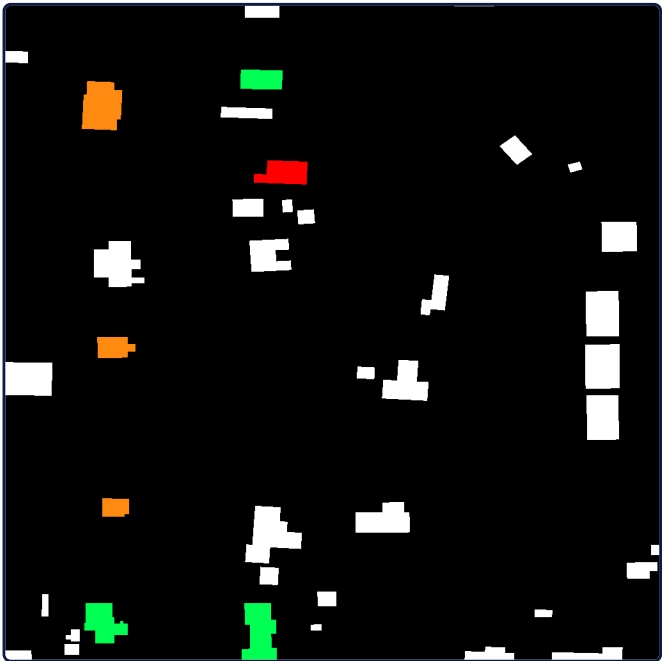
End-to-end deployment blueprint — county/state-scale USA · post-disaster imagery → damage map

- Siamese U-Net
- xBD / xView2
- Maxar STAC
- PostGIS + Vector Tiles
- H3 Aggregation
- Event-Triggered Batch

1 · Input → Output: What the Model Learns



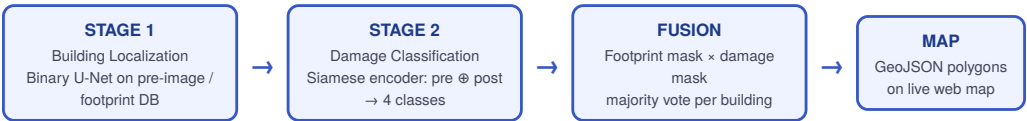
INPUT — Post-disaster aerial tile (Hurricane Michael, Oct 2018, NOAA ERI, ~30cm). Debris fields, downed trees, roof damage visible.



OUTPUT — Per-pixel segmentation mask. Every polygon = one building footprint, color = damage class predicted by the model.

Mask Color	Class ID	Damage Level	Visual Cue in Input Image
☐ White	1	No damage	Intact roofline, clean edges, no surrounding debris
☒ Green	2	Minor damage	Partial roof material loss, debris in yard
☒ Orange	3	Major damage	Structural roof failure, walls exposed
☒ Red	4	Destroyed	Collapsed structure, only foundation/debris pile
☒ Black	0	Background	Everything that is not a building (~97% of pixels)

2 · Approach & Stages



Damage is a **comparison problem**: a destroyed house in a post-only image is just a debris pile — the model must know a building stood there. Hence two stages, with the pre-disaster reference feeding stage 1 and stage 2.

⚠ THE PRE-DISASTER IMAGE PROBLEM (main deployment risk)

In deployment you often **won't have a fresh, aligned pre-disaster image**: archive imagery may be years old, from a different sensor/angle, cloud-covered, or simply not purchased. Three fallback tiers, in order of preference:

Tier	Strategy	How	Cost
A	Archive pre-image + co-registration	Pull nearest cloud-free pre-event scene from Maxar Open Data / Sentinel-2 archive; co-register to post-image (SIFT/LoFTR keypoints + homography). Works even if pre-image is 1–3 yrs old — buildings rarely move.	Best accuracy; adds alignment step
B	Footprint database as the "pre" signal	Skip pre-imagery entirely. Use Microsoft US Building Footprints / OpenStreetMap as ground-truth localization, rasterize footprints as an extra input channel, run a post-only damage classifier per footprint.	~3–5 F1 drop on minor vs major; zero imagery dependency
C	Single-image model	Train a post-only U-Net on xBD (ignore pre). Detects "destroyed" well (debris is obvious) but confuses no-damage vs minor.	Last resort / first-hours triage

Deploy A with automatic fallback to B. The system checks the STAC archive for a pre-scene; if none within tolerance, it switches to footprint-conditioned post-only mode and flags lower confidence on minor/major classes.

3 · Difficulties (ranked by real-world impact)

#	Difficulty		Mitigation
1	Pre-image unavailable / misaligned — off-nadir angle, old archive, sensor mismatch; buildings shift 5–20 px between pre/post	HIGH	Tiered fallback above + LoFTR co-registration + train with random pre/post misalignment augmentation
2	Imagery latency — satellites/aircraft fly 1–5 days post-event; clouds block optical for days after storms	HIGH	Event-triggered pipeline that polls STAC every 30 min; SAR (Sentinel-1) coarse layer for day-0
3	Domain shift — trained on Florida hurricane, deployed on CA wildfire / different sensor → large F1 drop	HIGH	Hold out entire disasters as test sets; multi-disaster training; test-time augmentation; geo-foundation-model encoder (Prithvi/SatMAE)
4	Class imbalance — background ≈97% of pixels, "destroyed" <2% of buildings	MED	Focal + Dice loss, oversample damaged tiles, per-class F1 metric (never pixel accuracy)
5	Scale — one county ≈ tens of thousands of 1024 ² tiles; buildings split at tile borders	MED	128 px tile overlap + prediction stitching; batched GPU inference (ONNX FP16, ~20 tiles/s on T4)
6	Georeferencing errors — 10 m geotag error puts every polygon on the wrong house	MED	Snap predictions to footprint DB; report per-building confidence
7	False alarms erode trust — a "destroyed" label on an intact home is costly	LOW	Confidence threshold + human review queue for low-confidence buildings

4 · End-to-End Deployment Architecture

① **TRIGGER** — NWS/USGS/FEMA alert webhook or manual event creation → defines AOI polygon + event date



② **INGESTION** — poll Maxar Open Data / NOAA ERI STAC APIs every 30 min for post-event scenes in AOI · query archive for nearest pre-event scene
(fallback tier B if none → load MS Building Footprints for AOI instead)



③ **PREPROCESSING** — reproject to EPSG:3857 · cloud mask · co-register pre ↔ post (keypoints + homography) · tile 1024×1024, stride 896 (128 px overlap) · normalize per-sensor stats



④ **INFERENCE** — Celery/Huey GPU workers · Stage-1 localization U-Net + Stage-2 Siamese damage U-Net (ONNX FP16) · logits saved per tile



⑤ **POSTPROCESSING** — stitch overlapping tiles (average logits) · argmax → mask · polygonize (rasterio) · per-building majority vote + confidence · H3 res-8 hex aggregation for dashboard stats



⑥ **SERVING** — GeoJSON → PostGIS → vector tiles (tippecanoe/Martin) → Leaflet/Mapbox GL web map, damage-colored buildings + hex heat layer + review queue

5 · Code — Ingestion (Maxar STAC → tiles)

```
# ingest.py – pull post/pre scenes for an AOI, tile to 1024px (pip install pystac-client rasterio shapely)
from pystac_client import Client
import rasterio, numpy as np
from rasterio.windows import Window

MAXAR = "https://maxar-opendata.s3.amazonaws.com/events/catalog.json" # static STAC catalog per event

def find_scenes(aoi_geojson, event_date, when="post"):
    cat = Client.open(MAXAR)
    dt = f"{event_date}/.." if when == "post" else f"../{event_date}"
    items = cat.search(intersects=aoi_geojson, datetime=dt).item_collection()
    # pre: newest cloud-free before event | post: earliest after event
    items = sorted(items, key=lambda i: i.datetime, reverse=(when == "pre"))
    return [i for i in items if i.properties.get("eo:cloud_cover", 0) < 20]

def tile_scene(cog_url, out_dir, size=1024, stride=896): # 128px overlap
    with rasterio.open(cog_url) as src: # COG = stream, no full download
        for y in range(0, src.height - size, stride):
            for x in range(0, src.width - size, stride):
                w = Window(x, y, size, size)
                img = src.read(window=w)
                if img.mean() < 5: continue # skip nodata tiles
                profile = src.profile | {"width": size, "height": size,
                                         "transform": src.window_transform(w)} # keeps georeference!
                with rasterio.open(f"{out_dir}/t_{x}_{y}.tif", "w", **profile) as dst:
                    dst.write(img)

scenes = find_scenes(aoi, "2024-09-27", "post")
pre = find_scenes(aoi, "2024-09-27", "pre") # [] → fallback tier B (footprints)
```

6 · Code — Training Config (Siamese U-Net on xBD)

```
# train_config.yaml + model core (pip install segmentation-models-pytorch albumentations)
# --- config ---
data: {root: ./xbd, tile: 1024, holdout_disasters: [hurricane-michael, santa-rosa-wildfire]}
model: {encoder: efficientnet-b4, weights: imagenet, classes: 5, siamese: true}
loss: {focal_gamma: 2.0, dice_w: 1.0, class_w: [0.05, 1, 4, 4, 6]} # bg heavily down-weighted
train: {epochs: 60, bs: 8, lr: 2e-4, opt: adamw, sched: cosine, amp: true}
aug: [hflip, vflip, rot90, color_jitter, pre_post_shift_20px] # simulate misalignment!

# --- model ---
import torch, segmentation_models_pytorch as smp

class SiameseUNet(torch.nn.Module):
    def __init__(self, enc="efficientnet-b4", n_cls=5):
        super().__init__()
        self.net = smp.Unet(enc, encoder_weights="imagenet",
                           in_channels=6, classes=n_cls) # pre(3) @ post(3) stacked
    def forward(self, pre, post):
        # tier-B mode: pre = rasterized footprint mask repeated to 3ch
        return self.net(torch.cat([pre, post], dim=1))

loss = lambda p, y: smp.losses.FocalLoss("multiclass", gamma=2)(p, y) \
    + smp.losses.DiceLoss("multiclass")(p, y)
# metric: xView2 score = 0.3*loc_F1 + 0.7*harmonic_mean(per-class damage F1) – never pixel acc
```

7 · Code — Mask → GeoJSON → Map Serving

```
# serve.py — vectorize predictions, load to PostGIS, serve map (pip install rasterio shapely fastapi)
import rasterio, json
from rasterio.features import shapes
from shapely.geometry import shape

CLS = {1: "no-damage", 2: "minor", 3: "major", 4: "destroyed"}
COLOR = {1: "#ffffff", 2: "#2ecc40", 3: "#ff851b", 4: "#ff2020"}

def mask_to_geojson(mask_tif, conf_tif, out="damage.geojson"):
    feats = []
    with rasterio.open(mask_tif) as m, rasterio.open(conf_tif) as c:
        arr, conf = m.read(1), c.read(1)
        for geom, val in shapes(arr, mask=arr > 0, transform=m.transform):
            g = shape(geom).simplify(1e-5) # transform → real lat/lon coords
            feats.append({"type": "Feature", "geometry": g.__geo_interface__,
                          "properties": {"damage": CLS[int(val)], "color": COLOR[int(val)],
                                         "confidence": round(float(conf[arr == val].mean()), 3),
                                         "review": bool(conf[arr == val].mean() < 0.6)}})
    json.dump({"type": "FeatureCollection", "features": feats}, open(out, "w"))

# load + tile: ogr2ogr -f PostgreSQL PG:"dbname=trriage" damage.geojson
#             tippecanoe -o damage.pmtiles -z16 damage.geojson

# --- Leaflet frontend (index.html core) ---
// L.geoJSON(data, { style: f => ({ color: f.properties.color, weight: 1, fillOpacity: .65 }) })
// .bindPopup(f => `${f.properties.damage} · conf ${f.properties.confidence}`).addTo(map);
// + H3 hex layer: count destroyed per hex → choropleth for county-level triage view
```

8 · Datasets & Where to Get Them

Dataset	Use	Link
xBD / xView2 — 850k buildings, pre+post, 4 damage classes, 19 disasters	Primary training + benchmark	xview2.org
Maxar Open Data — free hi-res pre/post for every major recent disaster (STAC)	Deployment imagery feed + pre-image archive (Tier A)	maxar.com/open-data
NOAA Emergency Response Imagery — ~30cm post-event aerial, US storms	US hurricane post-imagery (source of the input image above)	storms.ngs.noaa.gov
Microsoft US Building Footprints — 130M footprints	Tier-B pre-signal + polygon snapping	github.com/microsoft/USBuildingFootprints
OpenFEMA / FEMA damage assessments	Ground-truth validation of deployed predictions	fema.gov/about/openfema/data-sets
SpaceNet 8 — flood damage, buildings + roads	Flood transfer test (JalRakshak crossover)	spacenet.ai
Sentinel-2 / Sentinel-1 — free 10m optical + SAR archive	Pre-image fallback archive; SAR day-0 coarse layer	dataspace.copernicus.eu
RescueNet / CRASAR / IDA-BD — UAV & aerial damage sets	Domain-shift stress tests	github / IEEE DataPort (search names)
xView2 1st-place solution (Durnov) — reference code	Architecture + training recipe reference	github.com/DIUx-xView/xView2_first_place

✓ Build order

Phase 1 — Train Siamese U-Net on xBD (hold out 2 disasters) → ingest one historical event via Maxar STAC → GeoJSON on Leaflet map. Complete demo-able system. **Phase 2** — Tier-B footprint fallback + confidence/review queue + Huey worker queue + H3 dashboard. **Phase 3** — SAR day-0 layer, geo-foundation-model encoder, multi-event continuous operation.